

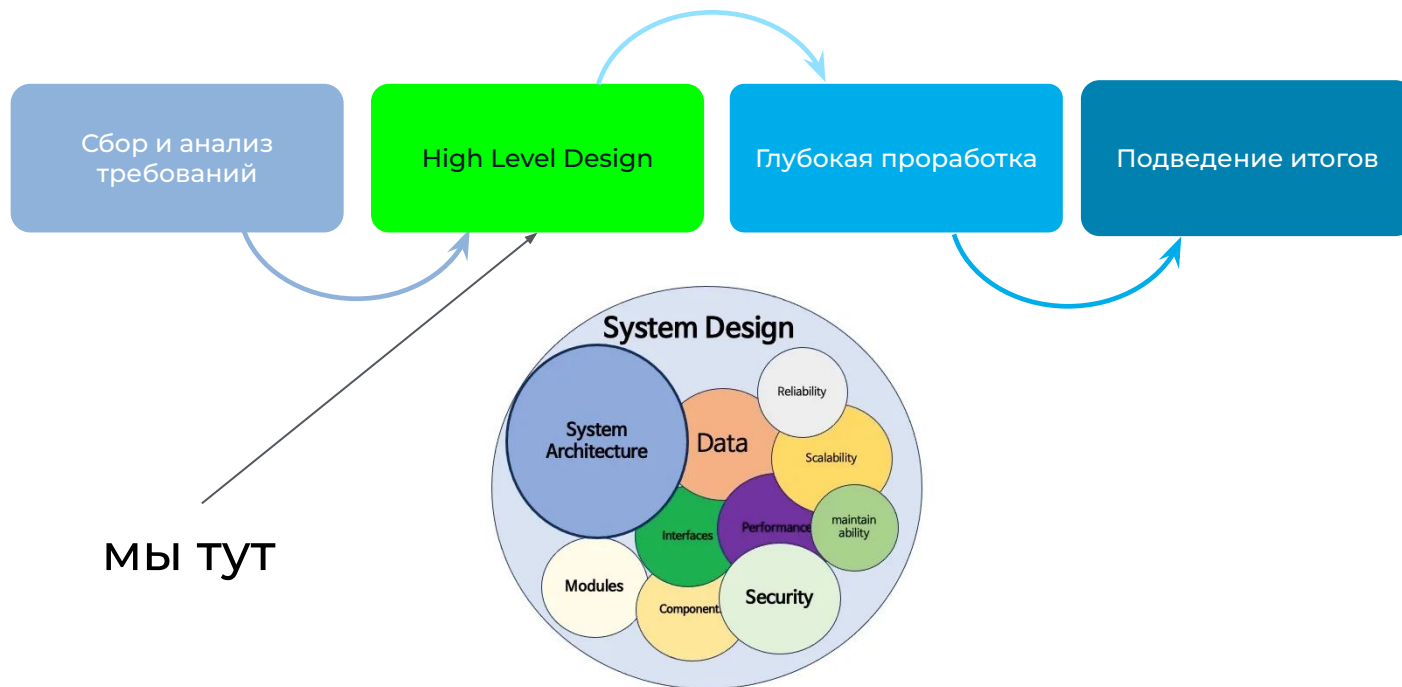


НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ

Системный дизайн современных приложений

Занятие №6
Интеграции. Часть 2.
Базы данных. Часть 1.

Этапность



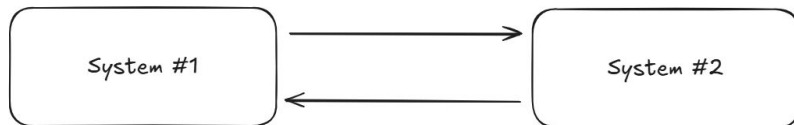


Алгоритм выбора

Сценарии

1. Получить данные на Frontend для отрисовки корзины товаров
 - а. Синхронное: хотим увидеть товары здесь и сейчас
2. Требуется гарантированная доставка документов
 - а. Асинхронное: можно подумать над ретраями, но лучше Kafka + DLQ
3. Генерация отчета
 - а. Асинхронное: долгая операция, сгенерироваться может и позже
4. Бизнес хочет, чтобы в браузере рендерилось уведомление (работа колокольчика)
 - а. Асинхронное: вебхуки или sse
5. Поточковая обработка данных, например, логирование
 - а. Асинхронное: не требуют мгновенной обработки
6. Перевод денег в онлайн-банке
 - а. Синхронное
7. Бронируем билеты в отель
 - а. Синхронное: нельзя допустить бронь одного места 2 раза

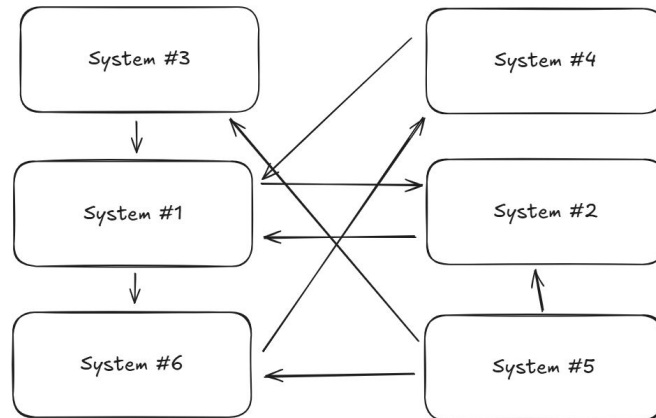
P2P (точка-точка, point-to-point)



N сервисов = до $N \times (N-1)/2$ прямых связей

2 сервиса = $2 \times 1/2 = 1$ связь

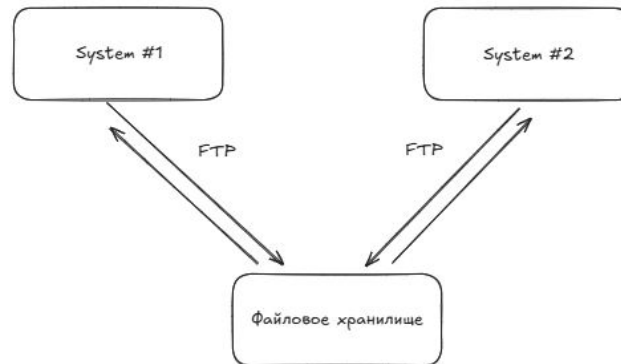
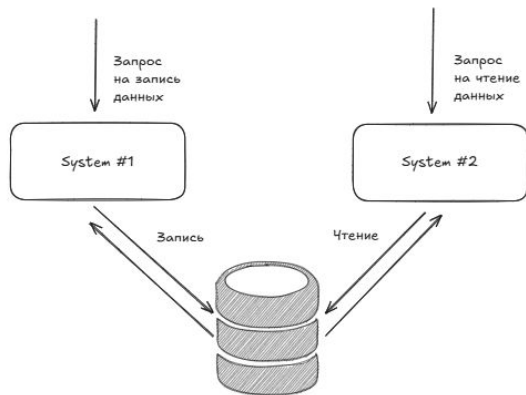
5 сервисов = $5 \times 4/2 = 10$ связей



Особенности

- Каждый сервис знает адреса других
- При изменении API в одном сервисе требуется обновление во всех вызывающих сервисах
- Нет центральной точки контроля, наблюдаемости, безопасности
- Спагетти-интеграция

Общая база данных и FTP

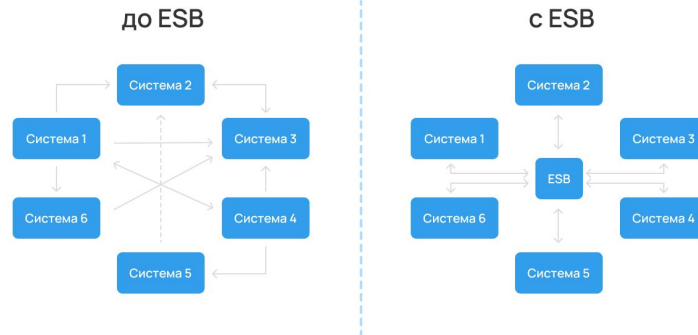


Особенности (БД)

- Плюс: не нужна новая инфраструктура, все просто и понятно
- Минусы:
 - Схема БД становится публичным контрактом между сервисами
 - Нет четкого владельца данных
 - Нагрузка одного сервиса влияет на другой
 - Невозможно масштабировать сервисы независимо

Особенности (FTP)

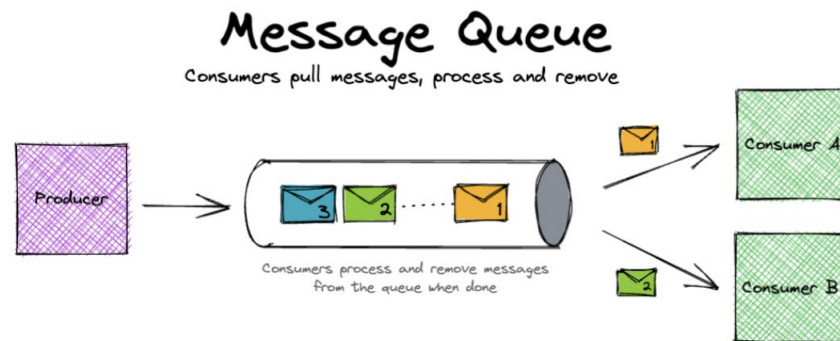
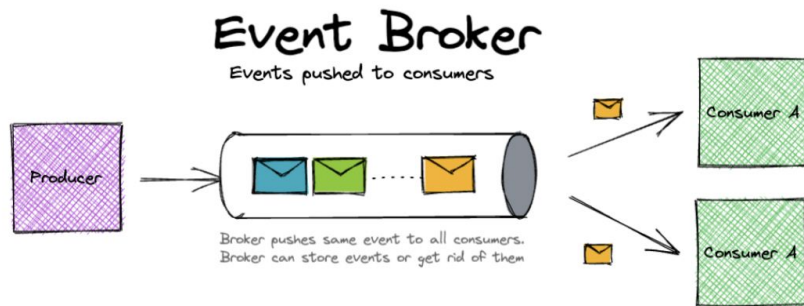
- Хорошо работает для больших объемов данных, пакетной обработки, интеграций с системами "умею только в файлы"
- Минусы:
 - Задержка: не real-time, а batch (час, день)
 - Обработка ошибок сложна: файл прочитан наполовину, что делать?
 - Формат файла: жесткий контракт, менять болезненно
 - Нет подтверждения обработки



Особенности

- Центральная шина, через которую идут все интеграции
- ESB умеет: маршрутизировать сообщения, трансформировать форматы (SOAP -> REST, XML -> JSON), оркестрировать бизнес-процессы, централизовать безопасность и мониторинг
- На практике:
 - ESB становился god object: вся бизнес-логика интеграций постепенно переезжала в шину
 - Единая точка отказа: все сообщения через один компонент
 - Дорого: лицензии IBM, Oracle, MuleSoft + поддержка
 - "Умная труба": вместо того чтобы быть тупым транспортом, ESB знал о бизнесе слишком много

Брокер сообщений / очередь сообщений



Особенности

- "Умная труба" для передачи сообщений, но без бизнес-логики
- Message Queue (RabbitMQ / ActiveMQ):
 - сообщение -> один получатель, потом удаляется
 - конкурирующие потребители делят нагрузку
- Message Broker (Kafka):
 - сообщение -> несколько получателей независимо
 - сообщения хранятся на брокере
 - потребитель сам отслеживает свою позицию

MQ

- Один producer кладет сообщение - один consumer его забирает. После обработки сообщение исчезает
- Каждое сообщение обрабатывается ровно одним consumer

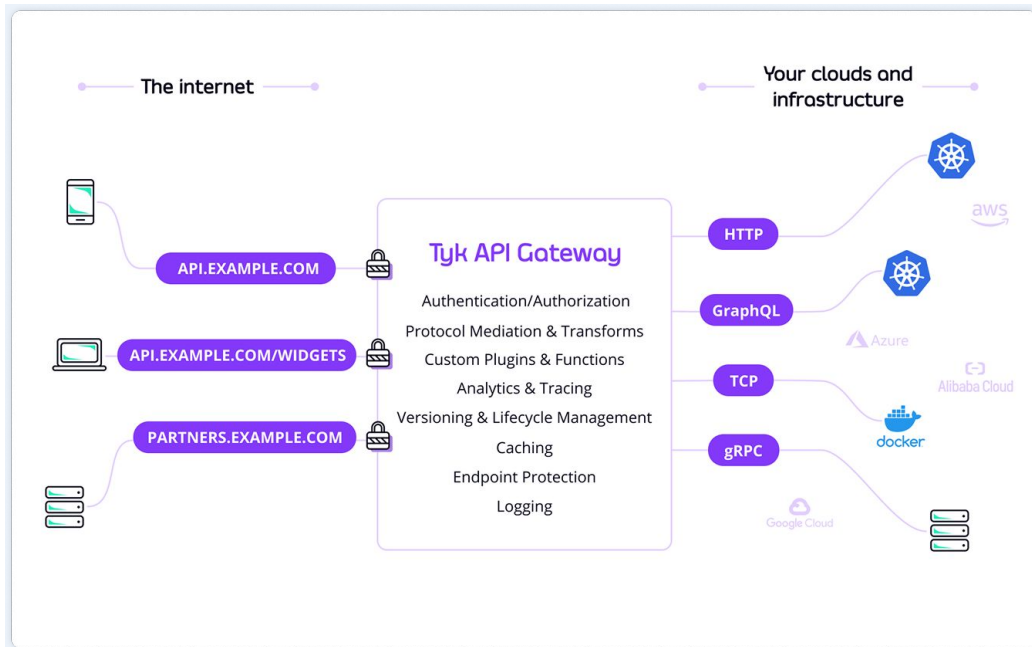
Broker

- Модель publish-subscribe. Producer публикует сообщение в топик - все подписчики получают его независимо
- Сообщение не исчезает после прочтения. Каждый consumer group читает независимо, отслеживая свою позицию (offset)

API GATEWAY

Особенности

- Появился как ответ на вопрос: как внешний клиент (мобильное приложение, система-потребитель) работает с десятками внутренних сервисов?
- Шлюз централизует: аутентификацию и авторизацию запросов, rate limiting и throttling (квоты), SSL терминацию, маршрутизацию, агрегацию нескольких вызовов в один, балансировку нагрузки, трансформацию данных, шифрование, паттерны отказоустойчивости, и т.д.
- Проблемы
 - SPOF (single point of failure)
 - Соблазн добавить бизнес-логику
 - Может быть bottleneck





REST

Что?

Архитектурный стиль взаимодействия компонентов распределенного приложения в сети
JSON over HTTP

Принципы REST

1. Клиент-серверная архитектура
2. Stateless
3. Кэширование
4. Единообразие интерфейса
5. Layered system
6. Code on demand

GET ▼ http://localhost:8001/items

Docs Params Authorization Headers (6) Body Scripts Settings

Body Cookies Headers (7) Test Results 🕒

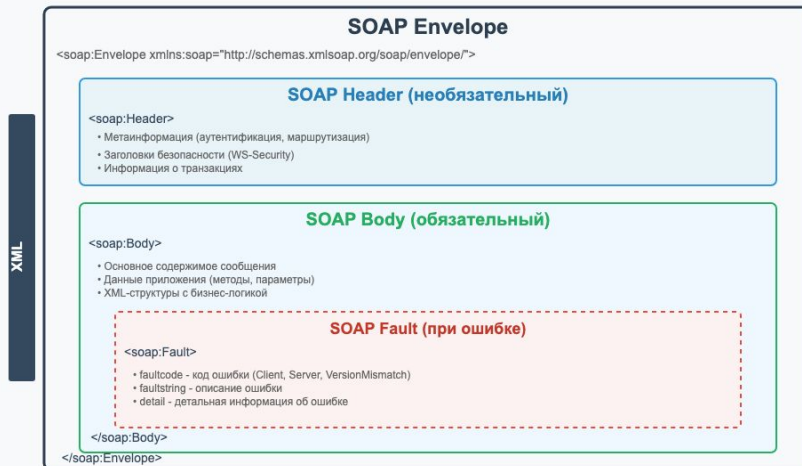
{} JSON ▶ Preview 🖼 Visualize ▼

```
1 {
2   "items": [
3     {
4       "id": 1,
5       "name": "Apple",
6       "_links": {
7         "self": "/items/1",
8         "delete": "/items/1"
9       }
10    },
11    {
12      "id": 2,
13      "name": "Banana",
14      "_links": {
15        "self": "/items/2",
16        "delete": "/items/2"
17      }
18    }
19  ],
20 }
```

 GET GET /users HTTP/1.1 Host: example.com Получает данные с сервера	 POST POST /users HTTP/1.1 Host: example.com Content-Type: application/json { "name": "Ivan" } Отправляет данные на сервер, создает новый ресурс	 PUT PUT /users/1 HTTP/1.1 Host: example.com Content-Type: application/json { "name": "Ivan Petrov" } Заменить существующий ресурс. Создать новый, при отсутствии	 PATCH PATCH /users/1 HTTP/1.1 Host: example.com Content-Type: application/json { "name": "Ivan" } Частично обновить ресурс
 DELETE DELETE /users/1 HTTP/1.1 Host: example.com Удалить ресурс с сервера	 HEAD HEAD /users HTTP/1.1 Host: example.com Получает только заголовки (без тела ответа)	 OPTIONS OPTIONS /users HTTP/1.1 Host: example.com Запрашивает возможности сервера и поддерживаемые методы	 CONNECT CONNECT example.com:443 HTTP/1.1 Host: example.com Устанавливается туннель к серверу
	 TRACE TRACE /users HTTP/1.1 Host: example.com Воспроизводит для проверки, возвращает полученный запрос		

	SOAP	REST
Рабочая область	Протокол. Использует XML для обмена сообщениями, определяет строгую структуру сообщений, включая заголовки и тело. Поддерживает транспортные протоколы, такие как HTTP, SMTP и другие.	Архитектурный стиль. Обычно использует JSON или XML для передачи данных, но может работать с любым форматом (например, HTML, текст). Работает в основном через HTTP и использует стандартные методы (GET, POST, PUT, DELETE). Более простой и легковесный подход.
Структура и сложность	Сложная структура с обязательными заголовками и использованием WSDL для описания интерфейса. Подходит для сложных операций и транзакций.	Ориентирован на ресурсы, где каждый из них имеет уникальный URL. Не требует строгого описания интерфейса.
Безопасность	Поддерживает WS-Security для безопасности сообщений. Может использовать сложные механизмы аутентификации и шифрования.	Обычно полагается на стандартные механизмы безопасности HTTP, такие как HTTPS. Может использовать OAuth для аутентификации.
Стандарты и расширяемость	Имеет множество стандартов — WS-ReliableMessaging, WS-Security и другие.	Более гибкий и расширяемый. Не зависит от стандартов.
Применение	Государственные и бизнес-приложения, где важны безопасность и надёжность (например, банковские системы или налоговая).	Веб и мобильные приложения с более простой аутентификацией. Например, где не нужны паспортные данные или реквизиты счёта. Чаще всего REST используют для мобильных игр.

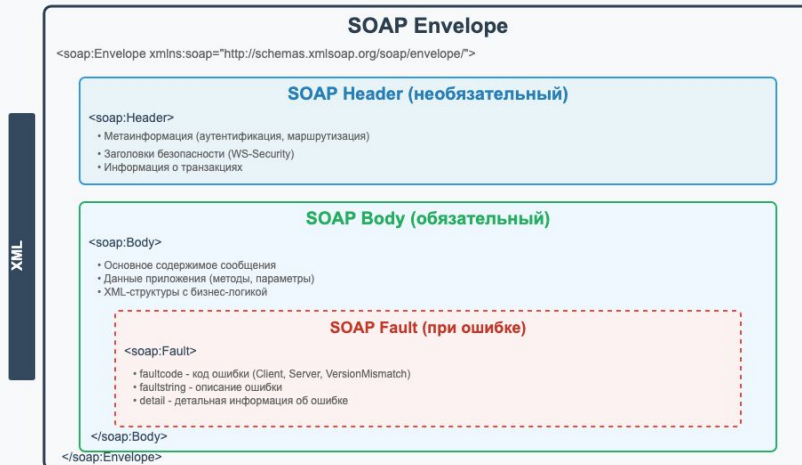
Структура SOAP-сообщения



SOAP v1.1/1.2 Protocol Structure

	SOAP	REST
Рабочая область	Протокол. Использует XML для обмена сообщениями, определяет строгую структуру сообщений, включая заголовки и тело. Поддерживает транспортные протоколы, такие как HTTP, SMTP и другие.	Архитектурный стиль. Обычно использует JSON или XML для передачи данных, но может работать с любым форматом (например, HTML, текст). Работает в основном через HTTP и использует стандартные методы (GET, POST, PUT, DELETE). Более простой и легковесный подход.
Структура и сложность	Сложная структура с обязательными заголовками и использованием WSDL для описания интерфейса. Подходит для сложных операций и транзакций.	Ориентирован на ресурсы, где каждый из них имеет уникальный URL. Не требует строгого описания интерфейса.
Безопасность	Поддерживает WS-Security для безопасности сообщений. Может использовать сложные механизмы аутентификации и шифрования.	Обычно полагается на стандартные механизмы безопасности HTTP, такие как HTTPS. Может использовать OAuth для аутентификации.
Стандарты и расширяемость	Имеет множество стандартов — WS-ReliableMessaging, WS-Security и другие.	Более гибкий и расширяемый. Не зависит от стандартов.
Применение	Государственные и бизнес-приложения, где важны безопасность и надёжность (например, банковские системы или налоговая).	Веб- и мобильные приложения с более простой аутентификацией. Например, где не нужны паспортные данные или реквизиты счёта. Чаще всего REST используют для мобильных игр.

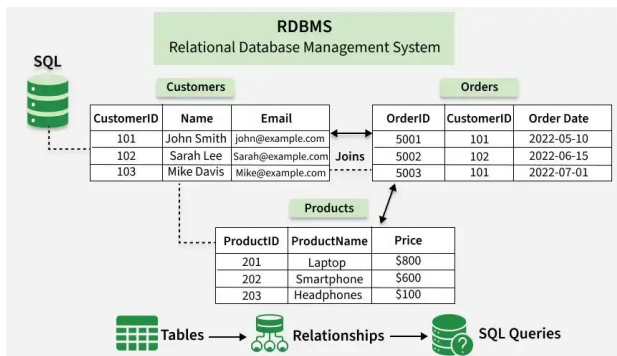
Структура SOAP-сообщения



SOAP v1.1/1.2 Protocol Structure

Базы данных





Filter connections by na

- demo jdbc:postgresql://localh...
- Databases
 - demo
 - Schemas
 - public
 - Tables
 - orders 29M
 - users 48K
 - Foreign Tables
 - Views
 - Materialized Views
 - Indexes
 - Functions
 - Sequences
 - Data types
 - Aggregate functions
 - Event Triggers
 - Extensions
 - Storage
 - System Info
 - Roles
 - Administer
 - System Info
 - HAPROXY LOCAL localhost:50...
 - HAPROXY REPLICA jdbc:postg...
 - MASTER YACLOUD rc1d-1g25l...

SQL Query:

```
SELECT id, user_id, amount, description, created_at
FROM public.orders;
```

Query Results:

	id	user_id	amount	A2 description	created_at
1	123	1	73.49	k6 load	2026-04-21 20:05:13.839 +0300
2	1,818,223	6	123	MY DESCRIPTION	2026-04-21 21:00:06.887 +0300

Value X: 1779726

200 row(s) fetched - 0.005s (0.002s fetch), on 2026-05-12 at 16:56:55

Транзакция

Что?

Упорядоченное множество операций, переводящих базу данных из одного согласованного состояния в другое: BEGIN TRANSACTION ... COMMIT

Зачем?

Решение проблем:

1. при сбоях БД
2. при конкурентном доступе до данных в БД

Когда НЕ нужны?

1. Читаем/обновляем только одну запись за операцию
2. Нет конкурентного доступа до данных





Транзакция

Что?

Набор гарантий безопасности, которые обеспечивают транзакции

Atomicity

Consistency

Isolation

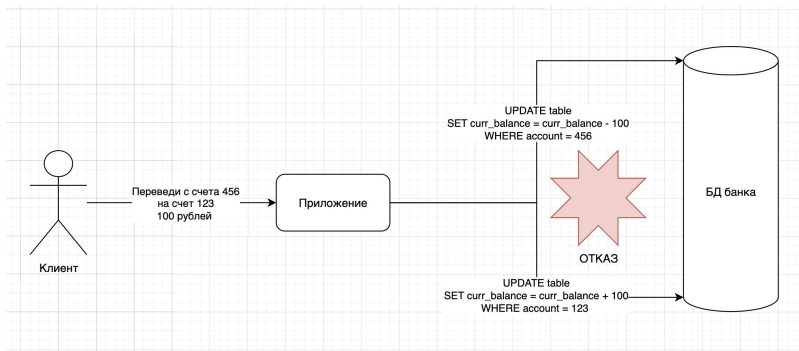
Durability

Атомарность (Atomicity)

Что?

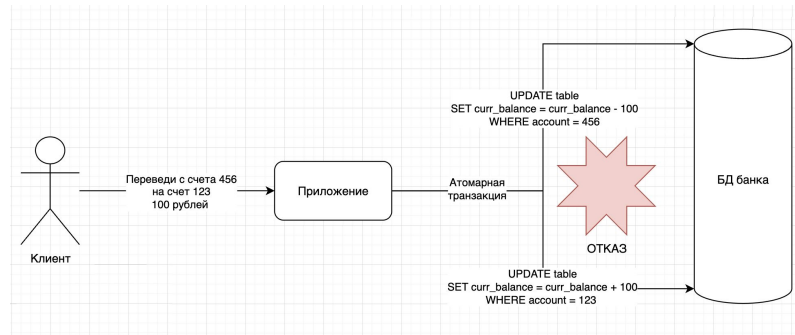
Транзакция будет выполнена:

- либо полностью (commit)
- либо не будет выполнена совсем (abort + rollback)



Итог

100 рублей растворились...



Итог

Вернули клиенту ошибку

Согласованность (Consistency)

Что?

- Атомарность, изоляция и сохраняемость - свойства базы данных, в то время как согласованность - свойство приложения
- По словам Клеппмана добавили в мнемонику для красоты



Про правильность данных

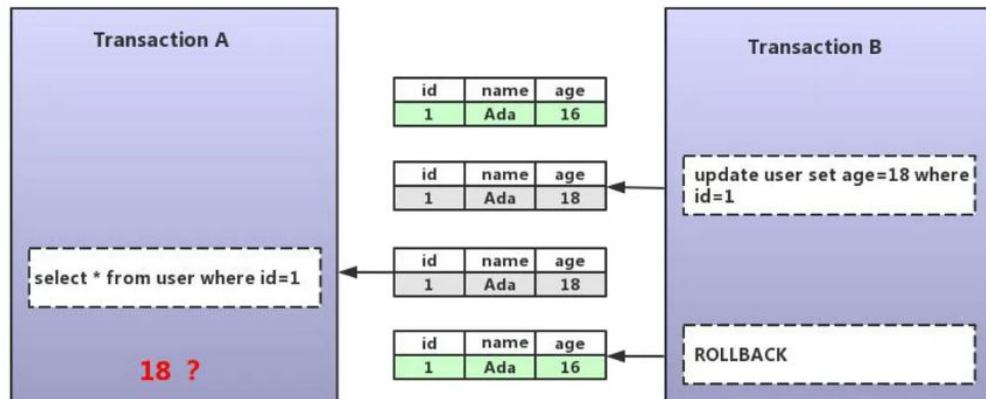
- Целостность данных (например, внешние ключи, уникальность, проверки CHECK)
- Бизнес-правила (например, «баланс счета не может быть отрицательным»)



Изоляция (Isolation)

Что?

Конкурентно выполняемые транзакции не могут мешать друг другу



Примеры

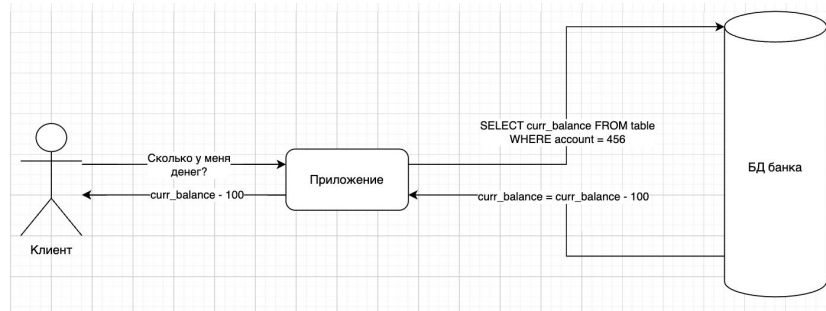
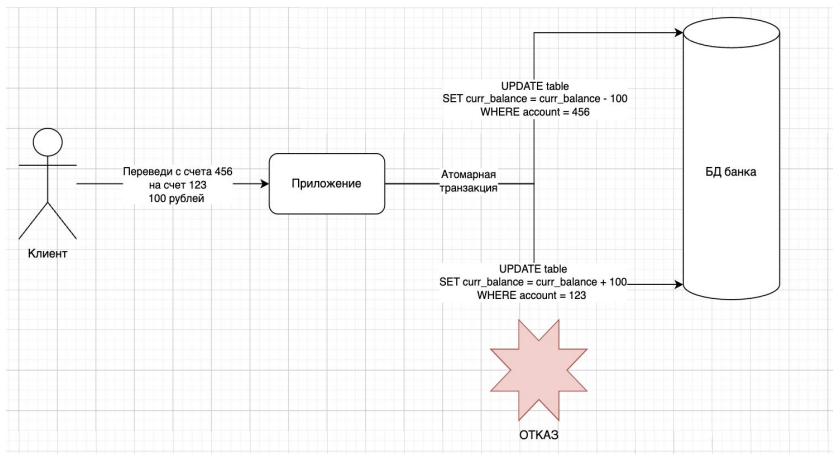
- Phantom Read: появились/исчезли строки между запросами
- Dirty Read: читаем незакоммиченные данные
- Non-Repeatable Read: данные изменились между чтениями

Что важно знать: Read Uncommitted / Read Committed / Repeatable Read / Serializable

Устойчивость (Durability)

Что?

Гарантия, что завершенные (зафиксированные) транзакции не потеряются



Флоу

1. Закоммитили транзакцию
2. БД упала

Флоу

3. БД восстановили
4. Данные актуальные

Что?

Класс СУБД, которые не являются реляционными
 NoSQL != No SQL
 NoSQL == Not Only SQL

Типы

1. + timeseries
2. + поисковые движки
3. + векторные

Зачем создали?

1. Потребность в распределенных СУБД
2. Новые модели данных
3. Работа с данными ускорилась
4. Избавление от ограниченности реляционных схем



1. Ключ-значение (Key-Value)



Данные хранятся в виде пары «ключ - значение». Простая структура и высокая производительность.

Примеры:

Redis, Amazon DynamoDB, Riak



Использование:
 кеширование, сессии, корзины покупок, профили пользователей

2. Документные БД (Document)



Данные хранятся в виде документов (например, JSON, BSON). Гибкая иерархическая структура.

Примеры:

MongoDB, CouchDB, Amazon DocumentDB



Использование:
 каталоги товаров, CMS, пользовательский контент, аналитика

3. Колонночные БД (Column-family)



Данные хранятся по столбцам внутри семейств колонок. Подходит для работы с большими объемами данных.

Примеры:

Apache Cassandra, HBase, ScyllaDB



Использование:
 логирование, временные ряды, IoT, аналитика больших данных

4. Графовые БД (Graph)



Данные представляются в виде узлов и связей между ними. Оптимизированы для сложных связей и отношений.

Примеры:

Neo4j, Amazon Neptune, JanusGraph



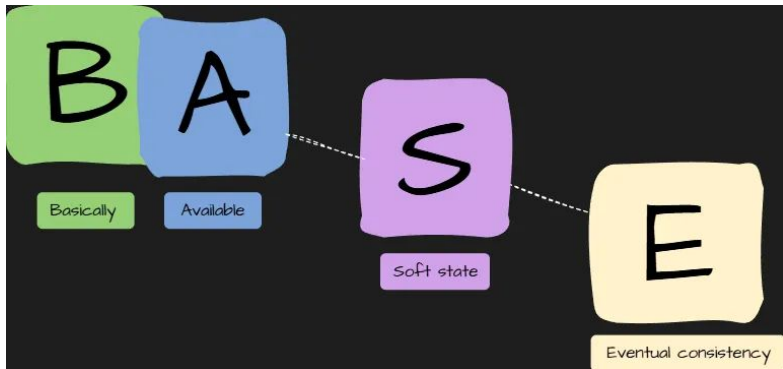
Использование:
 социальные сети, рекомендательные системы, мошенничество, сети связей

Вопрос

1 млн RPS

Транзакции блокируют запись

Что делать?



Что?

Модель транзакций для NoSQL

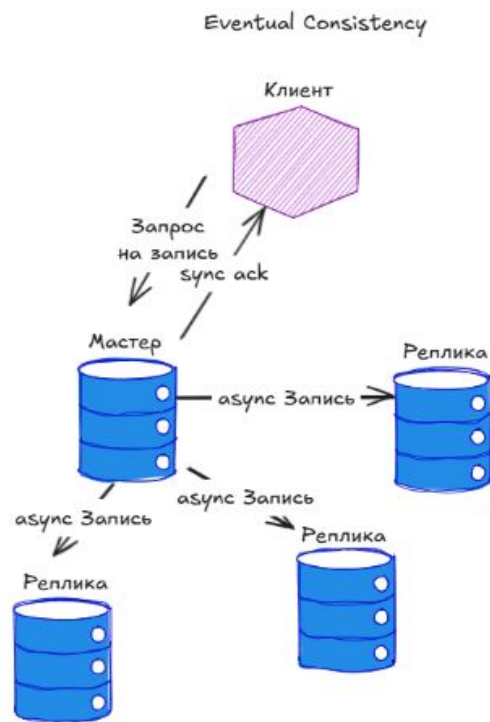
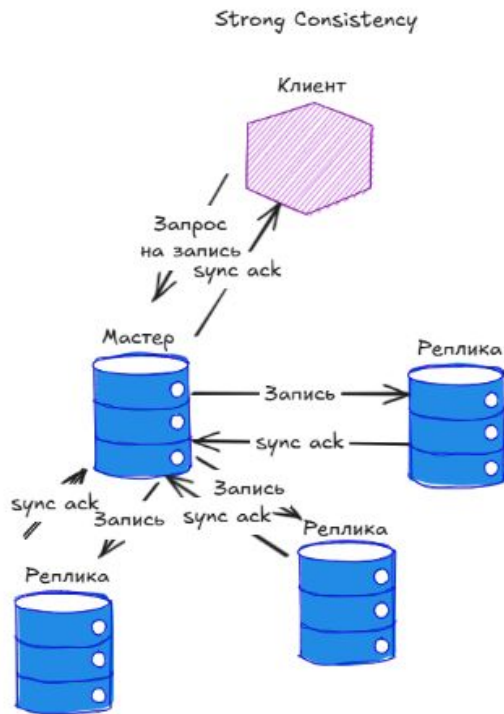
Basically Available: система всегда отвечает, даже с устаревшими данными

Soft state: данные могут временно быть несогласованными

Eventually consistent: согласованность достигается со временем

Виды согласованности

Характеристика	Strong	Eventual
Гарантия согласованности	Данные одинаковые на всех узлах	Разные узлы могут временно содержать разные версии данных
Задержка обновлений	Высокая – каждый запрос ждёт подтверждения от всех узлов	Низкая – изменения распространяются асинхронно
Пример использования	Банковские транзакции	Социальные сети, поисковые системы, кэширование
Инфраструктурные особенности	Требует синхронной репликации и блокировок	Использует асинхронные механизмы репликации



CAP-теорема

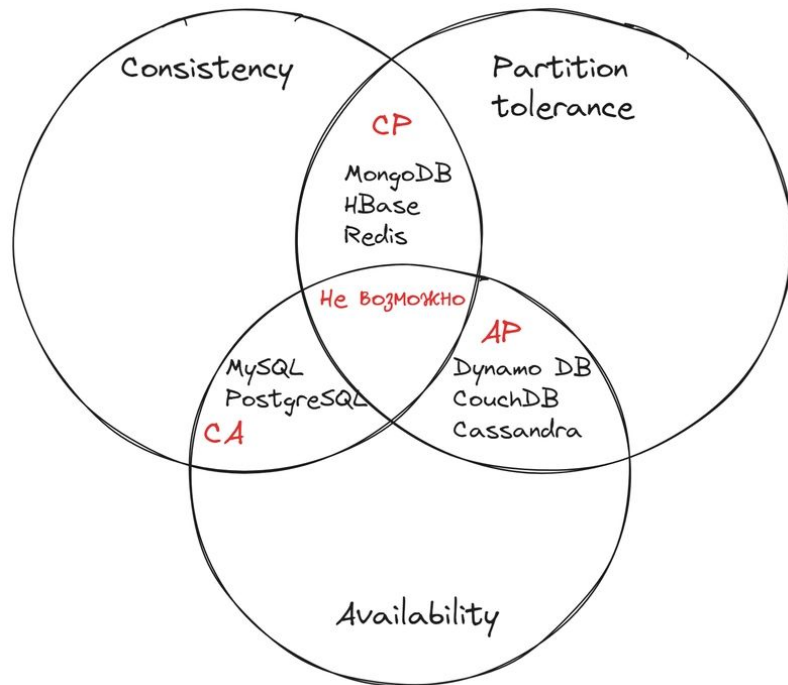
CAP теорема про компромиссы

Распределенные системы могут обеспечивать **только 2** свойства из 3-х:

- Согласованность (Consistency) (!= Согласованность в ACID (!)): *в любой момент времени на узлах одни и те же данные*
- Доступность (Availability): *система всегда отвечает на запросы*
- Устойчивость к разделению (Partition tolerance): *если между узлами нет связи, система работает*

Проблемы

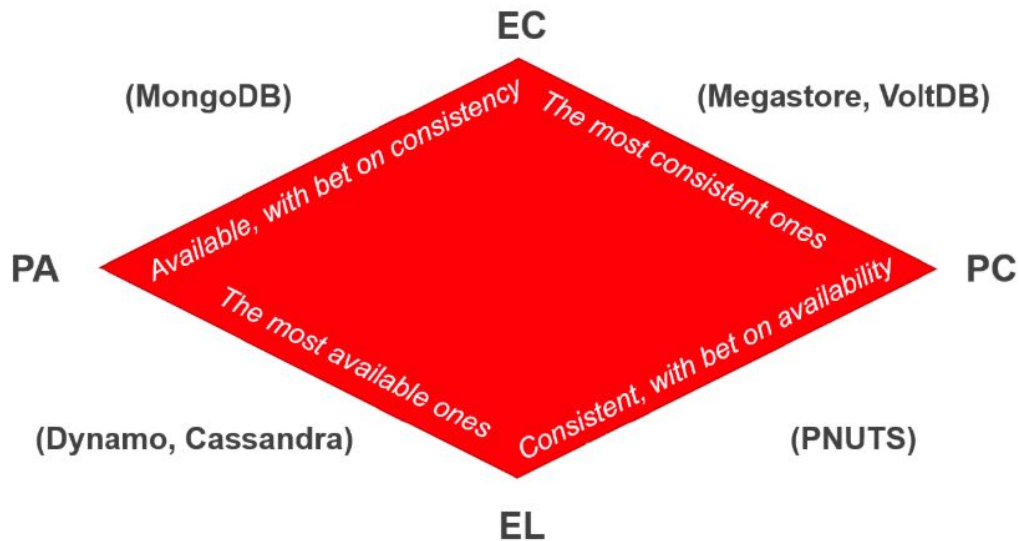
1. “Идеальный вольтметр”
2. В распределенных системах сетевые сбои неизбежны, а значит P - обязательно
3. Игнорирование задержек





PACELC-теорема

if **P**artition tolerance -> (**A**vailability or **C**onsistency) Else (**L**atency or **C**onsistency)





Репликация, шардирование, партиционирование



Horizontal scaling



Vertical scaling



Partition Strategies

Customer ID	First name	Last name	Favorite color
1	TAEKO	OHNASHI	blue
2	OLV	WRIGHT	green
3	SELDA	BAGGAN	purple
4	JIM	PEPPER	indigo

Vertical Partitions

Customer ID	First name	Last name
1	TAEKO	OHNASHI
2	OLV	WRIGHT
3	SELDA	BAGGAN
4	JIM	PEPPER

Horizontal Partitions

Customer ID	First name	Last name	Favorite color
1	TAEKO	OHNASHI	blue
2	OLV	WRIGHT	green
3	SELDA	BAGGAN	purple
4	JIM	PEPPER	indigo

Even distribution

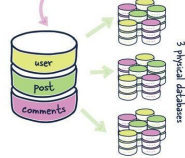


Uneven distribution



architecture notes

Monolith



Range Based Sharding

Product	Price
WIDGET	\$100
GIZMO	\$80
TRINKET	\$75
THINGAMAJIG	\$90
DOODAD	\$60
TOOTCHKE	\$99

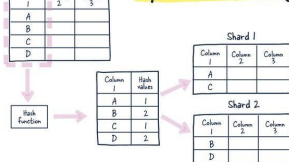
Product	Price
WIDGET	\$100
GIZMO	\$80
TRINKET	\$75
THINGAMAJIG	\$90
DOODAD	\$60
TOOTCHKE	\$99

Product	Price
WIDGET	\$100
GIZMO	\$80
TRINKET	\$75
THINGAMAJIG	\$90
DOODAD	\$60
TOOTCHKE	\$99

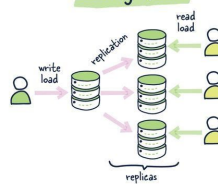
Shard Key

Column 1	Column 2	Column 3
A		
B		
C		
D		

Key Based Sharding



Scaling Reads





НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ

Репликация

Что?

Процесс копирования и синхронизации данных между несколькими узлами для обеспечения их доступности, надежности и высокой производительности

Виды

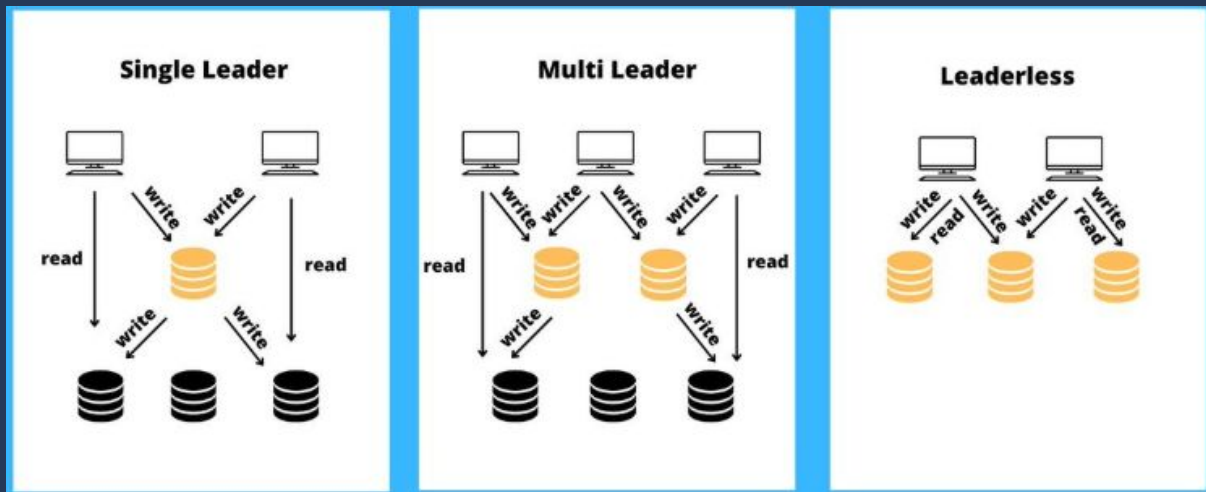
1. Single-Leader
2. Multi-Leader
3. Leaderless

Зачем создали?

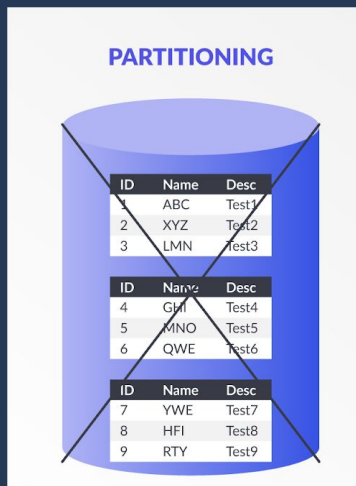
1. Повышение доступности
2. Улучшение производительности
3. Гео-распределение

Типы

1. Синхронная / Асинхронная И (в зависимости от контекста)
1. Логическая / Физическая



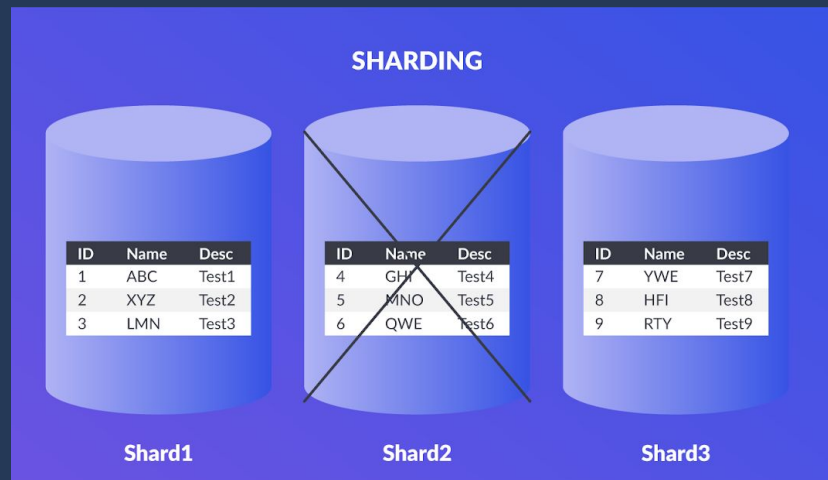
Шардинг и партиционирование



Партиционирование (Partitioning)

Логическое разделение данных внутри одной БД (например, по датам).

Цель: Ускорить запросы и упростить управление данными.

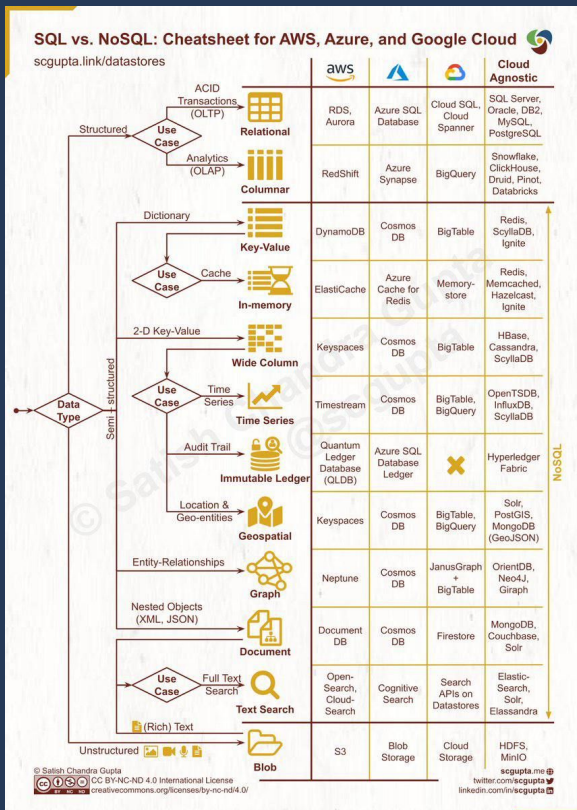


Шардинг (Sharding)

Физическое разделение данных между разными серверами (например, по user_id).

Цель: Распределить нагрузку на запись и хранить большие объемы данных.

Алгоритм выбора БД



Ряд вопросов

1. Что за данные?
2. Какой формат данных?
3. Какие отношения между данными? (ERD)
4. Какие сценарии использования данных?
5. Типовой ли проект?
6. Какие функции доступны из коробки?
7. Насколько популярна?
8. Какая лицензия?
9. Легко сопровождать и настраивать?
10. Что если данные изменятся?
11. (*) Есть ли сертификация/регистрация в ЕРРП?

<https://docs.google.com/viewerng/viewer?url=https://datafinder.ru/files/Cheat-Sheet-SUBD.pdf>